

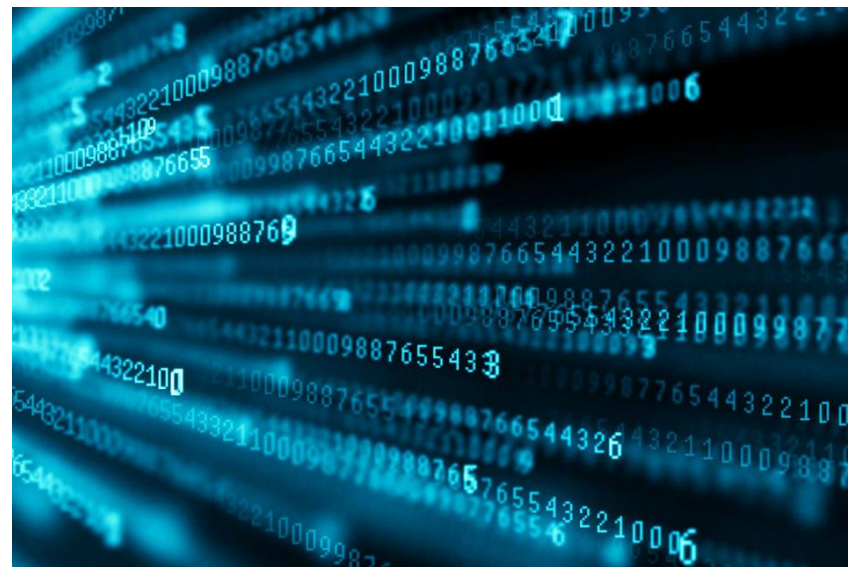
Data Stream I: Sampling

COMPSCI 753

The University of Auckland

Outline

1. Sampling from a Data Stream
 - Sampling a fixed proportion
 - Sampling a fixed-size (Reservoir Sampling)
2. Filtering a Data Stream: Bloom Filters
3. Finding Heavy Hitters in Data Stream
 - Misra-Gries Algorithm
 - CountMin Sketch Algorithm
 - Count Sketch Algorithm



Data as Infinite and Non-stationary

Uniformly Sampling from a Stream

- Motivation

- Since we cannot store the entire stream, we store a **uniform sample** of the stream to answer queries

- Goal

- Out of the large number of elements $a_1, \dots, a_m$, we choose a small number of elements to keep in memory
- Sample a single element x from the stream $a_1, \dots, a_m$ uniformly at random
- The probability of being the sample for each element is the same:

$$\Pr [a_i \text{ is the sample } x] = 1/m$$

- We draw a uniform sample over the stream, not over the universe **U!!**

Two Sampling Problems

- **Problem 1 (EASY)**

- Sample a **fixed proportion** of elements in the stream ($1/10$)
- The sample size is $m/10$ given the stream of m elements

- **Problem 2 (HARD)**

- Maintain a random sample of **fixed size** over an infinite stream
- The sample size is fixed, i.e., s for at any time k

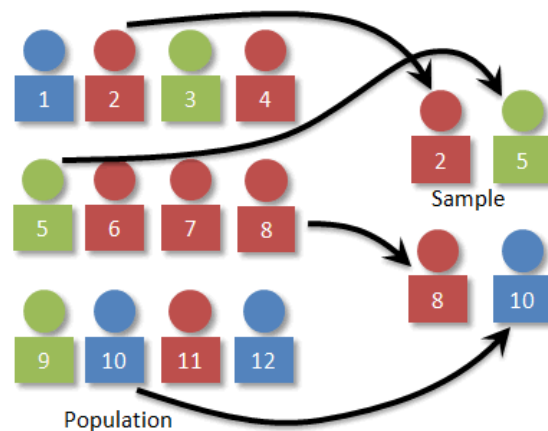
- Both problems: $\Pr [a_i \text{ is sampled }] = 1/m$

drawback : main memory limited



Sampling a Fixed Proportion

- Scenario
 - Search engine receives a stream of tuples (user, query, time)
 - **Question:** How **many users** run the same query in a single day?
 - **Memory:** We have enough space to store **1/10 of the stream**
- Naïve solution to keep 1/10 stream
 - Generate a random integer in $[0, \dots, 9]$ for each **tuple**
 - Store the **tuple** if the integer is 0; otherwise, discard
- We can approximately answer the question



Sampling 1/3 of the population

Problem with Naïve Approach

- Scenario:
 - Search engine receives a stream of tuples (user, query, time)
 - **Suppose:** Each user sends x queries once and d queries twice (total of $x + 2d$)
 - **Question:** What the fraction of queries by an average user are duplicates?
 - **Memory:** We have enough space to store $1/10$ of the stream
- Exact Answer: $d/(x + d)$

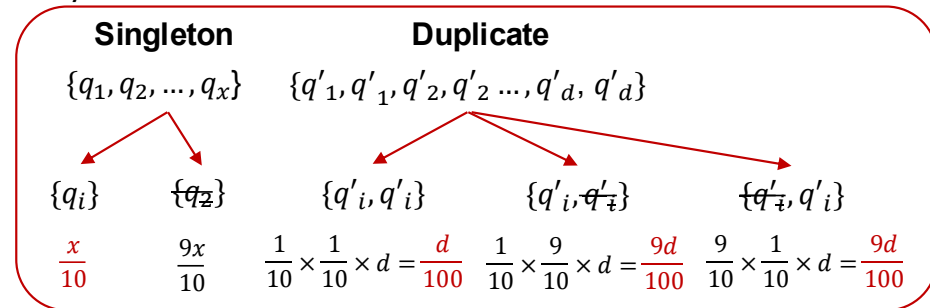
x
singleton queries
 $\{q_1, q_2, \dots, q_x\}$

$2d$
duplicate queries
 $\{q'_1, q'_1, q'_2, q'_2, \dots, q'_d, q'_d\}$
- **Claim:** we cannot approximately answer the questions using the naïve approach

Problem with Naïve Approach

- **Question:** What the fraction of queries by an average user are duplicates?
- Naïve solution to keep 10% of the tuples
 - Sample contains $x/10$ singleton queries
 - Sample contains only $d/100$ pairs of duplicates
 - A query is sampled once with prob. $1/10$
 - A query is sampled twice with prob. $1/10 \times 1/10 = 1/100$
 - Of d duplicates, $18d/100$ appear exactly once in the sample
 - A query is sampled at the first time with prob. $1/10$
 - A query is not sampled at the second time with prob. $9/10$
 - $((1/10 \cdot 9/10) + (9/10 \cdot 1/10)) \cdot d = 18d/100$

• **Sampled-based Answer:** $\frac{\frac{d}{100}}{\frac{x}{10} + \frac{d}{100} + \frac{18d}{100}} = \frac{d}{10x + 19d}$

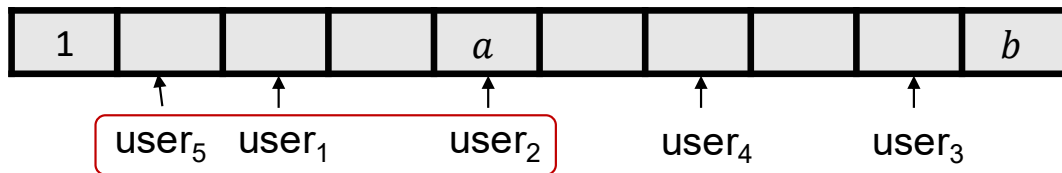


Solution: Sample Users

- Scenario:
 - Search engine receives a stream of tuples (user, query, time)
 - **Each user** sends x queries once and d queries twice (total of $x + 2d$)
- Question: What fraction of queries by an average user are duplicates?
- Solution:
 - Pick 1/10 of **users** (not tuples) and take all their searches in the sample
 - Use a **hash function** to hash the username (or user ID) uniformly into 10 buckets
 - Pick user IDs in the first bucket

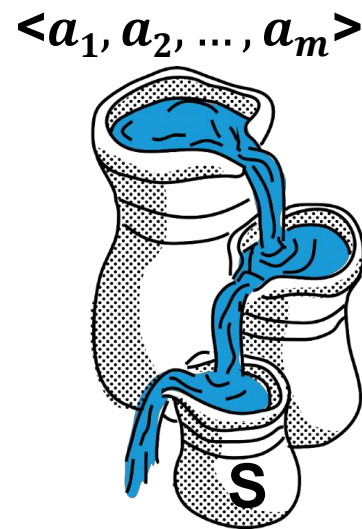
Generalized Solution: Fixed Proportion

- Stream of tuples with **keys**
 - Key is some subset of each tuple's components
 - E.g., Tuple is (user, query, time) → key is **user**
 - Choice of key depends on applications
- **Task:** Get a sample of a/b fraction of the stream (e.g., 30%)
 - Hash each tuple's key uniformly into b buckets (e.g., 10 buckets)
 - Pick the tuple if its hash value is in the first a bucket (e.g., the first 3 buckets)



Sample a Fixed Size

- **Challenge**
 - Have no idea how many unique items in the stream upfront
 - Uniformly sampling element from the stream
 - Items arrives rapidly one at a time
- **Goal:** Sample a set **S** with fixed size **s** regardless the length of the stream
- **Reservoir Sampling** (**s = 1**) [Vitter'85], key steps:
 - $s = a_1$
 - Replace s by a_2 with prob. $1/2$
 - Replace s by a_3 with prob. $1/3$
 - ...
 - Replace s by a_m with prob. $1/m$



← We have seen m items at time m , each item is in the sample **S** with equal prob. $1/m$

E.g., Reservoir Sampling of Stream $\langle a_1, a_2, a_3, a_4 \rangle$

Take a_1
as sample s ?

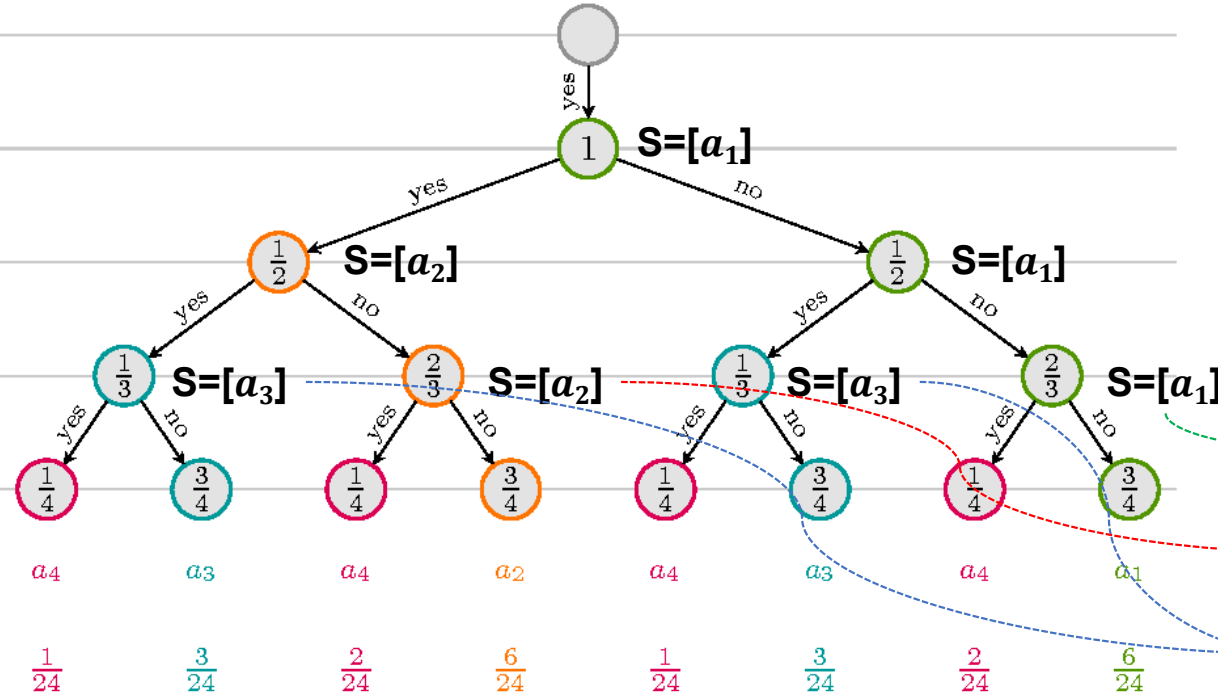
Take a_2
to replace s ?

Take a_3
to replace s ?

Take a_4
to replace s ?

The final
sample is

Probability
of this path



Q: What is the probability of element a_i picked as s after a_3 arrives?

A:

$$a_1: \frac{1}{2} \times \frac{2}{3} = \frac{1}{3}$$

$$a_2: \frac{1}{2} \times \frac{2}{3} = \frac{1}{3}$$

$$a_3: \frac{1}{2} \times \frac{1}{3} \times 2 = \frac{1}{3}$$

Reservoir Sampling: Proof

- For a stream $a_1, a_2, \dots, a_L, a_{L+1}, \dots, a_m$, we need to prove

$$\Pr[a_L \text{ is sampled}] = 1/m$$

- Proof**

- What is the probability that some a_L is the final s for $1 \leq L \leq m$?
- This happen if a_L is chosen as s and the rest $a_{L+1}, \dots, a_m$ are not chosen

$$\Pr[a_L \text{ is the final } s] = \Pr[a_L \text{ is chosen as } s] \cdot \prod_{i=L+1}^m \Pr[a_i \text{ does not replace } a_L \text{ as } s]$$

$$= \frac{1}{L} \cdot \prod_{i=L+1}^m \left(1 - \frac{1}{i}\right)$$

$$= \frac{1}{L} \cdot \prod_{i=L+1}^m \frac{i-1}{i}$$

$$= \frac{1}{L} \cdot \frac{L}{L+1} \cdot \frac{L+1}{L+2} \cdot \dots \cdot \frac{m-1}{m} = \frac{1}{m}$$

	$a_L, a_{L+1}, a_{L+2}, \dots, a_m$				
Chosen	$\frac{1}{L}$	$\frac{1}{L+1}$	$\frac{1}{L+2}$	$\dots$	$\frac{1}{m}$
Not-chosen		$1 - \frac{1}{L+1}$	$1 - \frac{1}{L+2}$	$\dots$	$1 - \frac{1}{m}$

Generalized Solution: Fixed Size

- **Reservoir Sampling** ($s > 1$) [Vitter'85]
 - Store all the first s elements of the stream to S
 - Suppose we have seen $m - 1$ elements, and now the m -th element arrives ($m > s$)
 - With prob. s/m , keep m -th element, else discard it
 - If we keep m -th element, then it replaces one of the s elements in S , picked uniformly at random
- **Claim:** After m elements, the sample S contains each element seen so far with the same probability s/m

Proof by Induction

- **Hypothesis:** After m ($m \geq s$) elements, $\Pr[a_L \text{ is sampled}] = s/m, 1 \leq L \leq m$
- **To prove:** When the $(m + 1)$ -th element arrives, we need to show

$$\Pr[a_L \text{ is sampled}] = s/(m + 1), 1 \leq L \leq m + 1$$

- Base Case:

First s elements, $\Pr[a_L \text{ is sampled}] = 1, 1 \leq L \leq s$

- Induction Step: $(m + 1)$ -th element a_{m+1} arrives:

- a_L already in S : $\Pr[a_L \text{ is sampled}] = s/m, 1 \leq L \leq m$ (hypothesis)

- The prob. that a_L is still in S is $\left(1 - \frac{s}{m+1}\right) + \frac{s}{m+1} \times \frac{s-1}{s} = \frac{m}{m+1}$

- $\Pr[a_L \text{ is sampled}] = \left(\frac{s}{m}\right) \times \left(\frac{m}{m+1}\right) = \frac{s}{m+1}, 1 \leq L \leq m$

Before process a_{m+1}

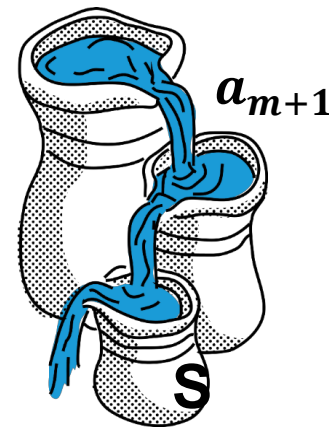
Process a_{m+1}

Case1: a_{m+1} is not chosen

Case2: a_{m+1} is chosen

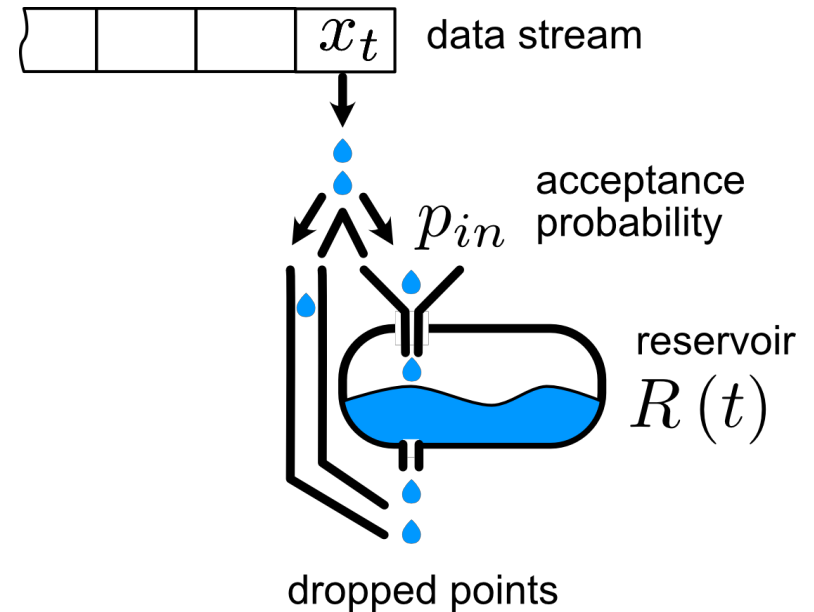
Case2: replace one element **other than** a_L in S

Case1: discard a_{m+1}
Case2: take a_{m+1} in S



Summary

- Sampling a fixed proportion
 - Naïve sampling approach
 - Hashing-based sampling approach
- Sampling a fixed-size
 - Reservoir sampling
 - Theoretical Proof



Source: <https://towardsdatascience.com/reservoir-sampling-for-efficient-stream-processing-97f47f85c11b>

Resources

- **[Book]** Mining of Massive Datasets [Book] by Jure Leskovec, Anand Rajaraman, and Jeff Ullman. Chapter 4
 - <http://mmds.org>
- **[Book]** Machine Learning for Data Streams with Practical Examples in MOA (<https://moa.cms.waikato.ac.nz/book-html/>) – Chapter 4